

Construindo seu Chat de IA Personalizado

Parte 3: Front-end com IA e Produção

Interface própria em React, API com FastAPI, PDF na base e deploy robusto no Railway, com o mesmo portão de qualidade

Parte 3 de 3 · 08/10 · Apostila do aluno

Prof. Thiago Azeredo Rodrigues

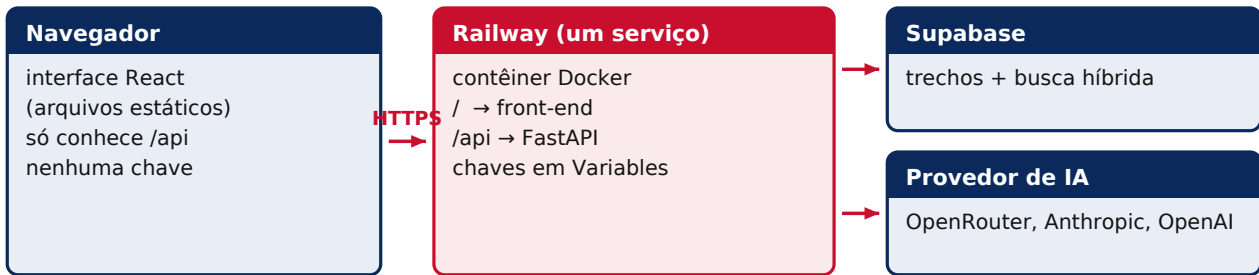
Parte 1 (24/09): CI/CD e Deploy

Parte 2 (01/10): RAG e Base de Conhecimento

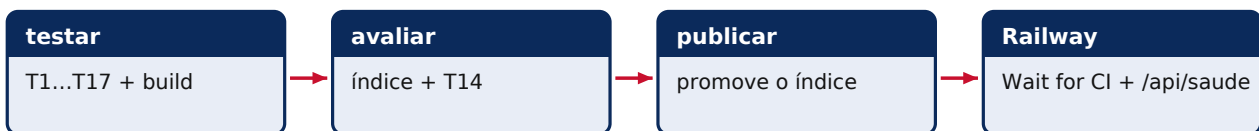
01 O que você vai construir hoje

Nas Partes 1 e 2, o assistente ganhou deploy automático e uma base de conhecimento. Ele funciona, mas tem cara de protótipo: a interface é a padrão do Gradio e ele mora num Space do Hugging Face. Hoje ele ganha **cara de produto**: uma interface própria, bonita e responsiva, servida por uma API num serviço de produção, com endereço próprio e HTTPS. E a base de conhecimento passa a aceitar **PDF**.

O que não muda é o mais importante: o portão. O código continua sendo testado e a busca continua sendo avaliada antes de qualquer coisa ir para o ar. Muda o destino, não o fluxo.



GitHub Actions: o portão continua decidindo o que vai para o ar



Item	Parte 2	Parte 3
Interface	Gradio, padrão	React + Tailwind, própria, com cartões de fonte, tema escuro e celular
Servidor	app.py dentro do Space	FastAPI no Railway: API + front-end no mesmo endereço
Endereço	huggingface.co/spaces/...	<servico>.up.railway.app, com HTTPS (ou domínio próprio)
Documentos	Só .md	.md e .pdf, com conversão automática
Deploy	Actions envia ao Space	Actions testa; o Railway só publica se tudo passar (Wait for CI)
O que trazer	Supabase e documentos	Conta no Railway e um PDF com texto da sua área

02 Do protótipo à produção

"Produção" não é um lugar, é um conjunto de garantias. Um protótipo prova que a ideia funciona; um sistema em produção precisa funcionar para os outros, todo dia, sem você olhando. As garantias que importam aqui:

Garantia	Pergunta que responde	Como resolvemos
Disponibilidade	Está no ar quando alguém acessa?	Serviço sempre ligado, com reinício automático em caso de falha

Garantia	Pergunta que responde	Como resolvemos
Reprodutibilidade	O que roda no servidor é o que eu testei?	Contêiner Docker: a mesma imagem no computador e no Railway
Controle do deploy	Quem decide o que vai para o ar?	O portão no GitHub Actions; o Railway espera o CI passar
Verificação	Como sei que o deploy deu certo?	Health check: a versão nova só entra se /api/saude responder
Segurança	As chaves estão protegidas? Dá para abusar?	Chaves só no servidor; limites de uso
Observabilidade	Quando quebrar, como descubro o porquê?	Logs do Railway e mensagens de erro claras
Reversão	Se a versão nova estiver ruim, como volto?	Rollback com um clique no Railway; o índice anterior continua no Supabase

Por que sair do Hugging Face agora

O Space foi a escolha certa para aprender: grátis, com GPU sob demanda e o Gradio pronto. Para um produto, três limites pesam: a interface fica presa ao Gradio, o push recusa binários (por isso PDF não entrava) e a URL é a do Hugging Face. O Railway é uma plataforma de aplicações (PaaS) mais próxima do que se usa em empresa: roda qualquer contêiner, tem domínio, health check, logs e rollback.

Plataforma	Bom para	Cuidados
Hugging Face Spaces	Demos de IA, Gradio, GPU	Interface e URL presas; binários recusados
Render	APIs e sites com contêiner, deploy por Git ou hook	No plano de 512 MB, o build também fica limitado a 512 MB (veja a seção 10)
Railway	Contêineres a partir do GitHub, build em máquina separada	Preço por uso (CPU e memória por minuto)
Fly.io	Contêineres em várias regiões	Configuração por linha de comando
Vercel, Netlify	Front-ends estáticos e funções curtas	Pouco adequados para servidor com modelo carregado
Azure App Service, Cloud Run, AWS	Produção corporativa	Mais configuração (rede, identidade, custos)

03 Front-end, back-end e a API entre eles

Até aqui, o Gradio escondia uma divisão que agora fica explícita. O **front-end** é o que roda no navegador do visitante: HTML, CSS e JavaScript. O **back-end** é o que roda no servidor: a busca, as chaves, a chamada ao modelo. Os dois conversam por uma **API**: um conjunto de endereços (rotas) que o servidor responde.

A regra que organiza tudo

Tudo que vai para o navegador é público: qualquer visitante pode abrir as ferramentas do navegador e ler o código. Por isso a interface nunca guarda chave, prompt de sistema nem nome de modelo. Ela manda a pergunta e recebe a resposta. É a mesma lição da Parte 1 sobre o GitHub Pages, agora aplicada ao nosso próprio front-end.

HTTP em uma tabela

Conceito	O que é	No projeto
Método	O tipo do pedido: GET lê, POST envia dados	GET /api/config; POST /api/perguntar
Rota	O endereço dentro do servidor	/api/saude, /api/config, /api/perguntar
Corpo (JSON)	Os dados enviados ou recebidos, em texto estruturado	{"mensagem": "...", "historico": [...]}
Código de status	O resultado: 2xx deu certo, 4xx erro de quem pediu, 5xx erro do servidor	200, 413 (grande demais), 422 (inválido), 429 (muitas perguntas), 503 (indisponível)
Cabeçalho	Informações extras do pedido ou da resposta	Content-Type, Retry-After, X-Forwarded-For

O contrato da API

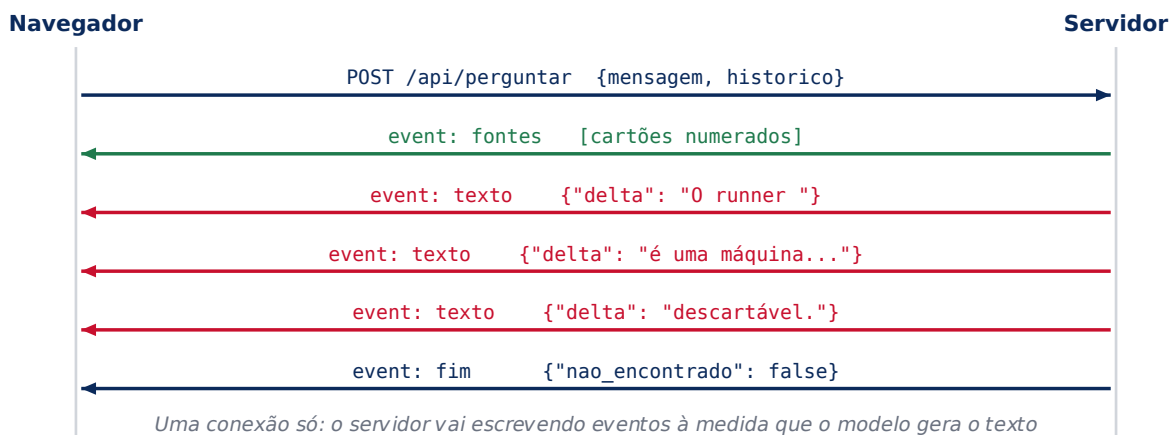
A spec define cada rota antes do código: o que recebe, o que devolve e quais erros. Esse contrato permite que o front-end e o back-end sejam construídos e testados separadamente, e é o que os testes automáticos conferem.

```
GET /api/saude      → {"status": "ok", "versao": "<commit>", "trechos_na_producao": 136}
GET /api/config    → {nome, descricao, exemplos, logo_url, cores, interface} (sem segredos)
POST /api/perguntar ← {"mensagem": "...", "historico": [...]}
                  → text/event-stream: fontes, texto, texto, ..., fim
```

04 Streaming: a resposta aos poucos

Um modelo de linguagem gera a resposta pedaço por pedaço. Esperar o texto inteiro para mostrar deixa o usuário olhando para uma tela parada por segundos. Streaming é entregar cada pedaço assim que ele sai. Há três jeitos comuns de fazer isso na web:

Técnica	Como funciona	Quando usar
Server-Sent Events (SSE)	Uma resposta HTTP que fica aberta e recebe eventos de texto, só do servidor para o navegador	Chat com IA: é o que as APIs de modelos usam. O nosso caso
WebSocket	Canal aberto nos dois sentidos	Jogos, colaboração em tempo real, chat entre pessoas
Polling	O navegador pergunta de tempos em tempos se há novidade	Atualizações lentas; desperdiça requisições



Repare na ordem: as **fontes** chegam primeiro, porque a busca acontece antes da geração. O front-end já pode preparar os cartões e só mostrá-los no fim, se a resposta não for "não encontrei".

05 Front-end moderno: React, Vite e Tailwind

React: a interface em componentes

No React, a tela é montada com **componentes**: funções que recebem dados e devolvem um pedaço de interface. Quando o **estado** muda (uma mensagem nova, um pedaço de texto que chegou), o React redesenha só o que mudou. A interface do projeto fica dividida assim:

Componente	O que mostra	Estado que usa
Cabecalho	Logo, nome, descrição, botão de tema	configuração vinda de /api/config
Sugestoes	As perguntas de exemplo na tela inicial	se a conversa está vazia
Conversa e Mensagem	Os balões, com Markdown e botão de copiar	lista de mensagens
CartaoFonte	fonte > seção; ao clicar, o trecho usado	aberto ou fechado
CaixaPergunta	Campo de texto, contador, botão de enviar	texto digitado, se está respondendo
AlternarTema	Claro ou escuro	preferência salva no navegador

Vite: do código ao site

O navegador não entende JSX nem Tailwind diretamente. O **Vite** faz duas coisas: em desenvolvimento, um servidor local que recarrega a tela a cada alteração (`npm run dev`); para publicar, um **build** (`npm run build`) que gera a pasta `dist/` com HTML, CSS e JavaScript prontos e compactados. É essa pasta que o servidor entrega.

```

cd frontend
npm install      # instala as dependências (package.json)
npm run dev     # http://localhost:5173, recarrega sozinho; /api vai para a porta 8000
npm run build   # gera frontend/dist, o que vai para produção
  
```

Tailwind: estilo sem escrever CSS do zero

Tailwind é uma biblioteca de classes utilitárias: em vez de criar um arquivo de estilo, você descreve o visual na própria marcação, como `rounded-2xl bg-white shadow-sm dark:bg-slate-900`. Ela facilita manter espaçamentos, cores e o modo escuro consistentes, e é um formato que os assistentes de IA geram bem.

06 Interface de chat que parece produto

A diferença entre "funciona" e "parece produto" está nos detalhes. Uma lista do que a spec pede e por quê:

Detalhe	Por que importa
Estado vazio com sugestões	Ninguém sabe o que perguntar diante de uma tela em branco
Indicador de "pensando"	Entre o envio e o primeiro pedaço há a busca e a fila do modelo; sem sinal, parece travado
Resposta em Markdown	Listas, negrito e código ficam legíveis, não um bloco de texto
Cartões de fonte expansíveis	Mostram de onde veio a resposta e deixam o usuário conferir: é a confiança do RAG visível
Erro com "tentar de novo"	Falhas acontecem (modelo lotado, rede); o usuário não pode perder a pergunta
Aviso no limite de uso	Dizer quanto tempo esperar é melhor que um erro genérico
Celular e modo escuro	Boa parte do acesso é pelo celular, e muita gente usa tema escuro
Acessibilidade	Contraste, teclado e rótulos: quem usa leitor de tela também precisa conseguir

Hierarquia visual em três camadas

- **Cores de marca** com moderação: a cor principal do `config.yml` no cabeçalho, no botão de enviar e nos detalhes; o resto em tons neutros.
- **Tipografia** com no máximo dois pesos e tamanhos consistentes: a resposta precisa ser confortável de ler em parágrafos longos.
- **Espaço**: margens generosas e largura máxima de leitura (cerca de 70 caracteres por linha) valem mais que qualquer efeito.

07 Gerando o front-end com IA

Assistentes de programação geram interfaces muito boas quando recebem um pedido preciso, e interfaces genéricas quando o pedido é vago. A spec cumpre esse papel: lista os componentes, os estados, as cores e o contrato da API. Para o visual, acrescente referências concretas:

Implemente a tarefa 3 da SPEC-parte3. Visual: interface de chat limpa e moderna, inspirada nos chats comerciais. Largura máxima de leitura de 768 px, centralizada. Cor principal vinda de `/api/config`. Cantos arredondados, sombras suaves, tons neutros (slate) no fundo, modo escuro com `dark:` do Tailwind. Mensagens do usuário alinhadas à direita na cor principal; do assistente à esquerda, sem balão, em Markdown. Cartões de fonte em grade abaixo da resposta. Mostre como testar.

O que revisar no que a IA gerou

- **Dependências:** cada pacote novo em package.json é código de terceiros no seu site. Pergunte para que serve cada um e prefira poucos.
- **Segredos:** procure `import.meta.env` e `VITE_`. Nada de chave ali (o T17 também confere).
- **Rotas:** o front-end deve chamar só caminhos relativos (`/api/...`), nunca a API do provedor de IA diretamente.
- **Estados:** teste de propósito o erro (desligue o servidor), o 429 (pergunte rápido) e o "não encontrei".

08 O servidor: FastAPI

FastAPI é um framework Python para APIs. Cada rota é uma função; os dados de entrada são descritos com **Pydantic**, que valida tudo antes de a função rodar (por isso uma pergunta vazia já volta com 422, sem código extra). O que o `app.py` fazia com o Gradio é dividido em duas partes: `chat.py`, com os provedores e a geração da resposta, e `servidor.py`, com as rotas.

```
class Pergunta(BaseModel):
    mensagem: str = Field(min_length=1)
    historico: list[Mensagem] = []

@app.post("/api/perguntar")
def perguntar(dados: Pergunta, request: Request):
    verificar_limites(dados.mensagem, ip_do_visitante(request)) # 413 ou 429
    return StreamingResponse(eventos(dados), media_type="text/event-stream")

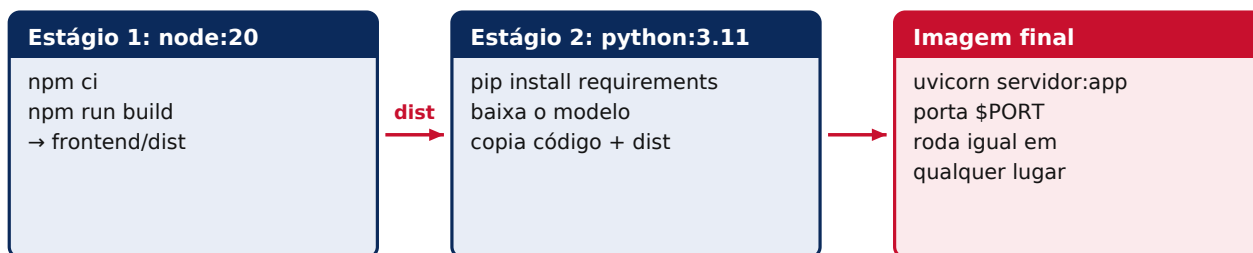
app.mount("/", StaticFiles(directory="frontend/dist", html=True)) # o front-end
```

A última linha faz o servidor entregar também o front-end compilado. Uma URL só para tudo: sem configuração de CORS (a regra do navegador que bloqueia chamadas entre endereços diferentes) e sem segundo serviço para manter.

09 Contêineres e Docker

A Parte 1 começou com a frase "na minha máquina funciona". O contêiner é a resposta mais completa a ela: em vez de mandar o código e torcer para o servidor ter as mesmas versões, você manda uma **imagem** com o sistema, o Python, as bibliotecas, o modelo de embedding e o código, tudo junto. A imagem roda igual no seu computador, no GitHub Actions e no Railway.

Termo	O que é
Dockerfile	A receita da imagem: um passo por linha (base, instalar, copiar, iniciar)
Imagem	O resultado da receita: um pacote imutável e versionado
Contêiner	Uma imagem em execução
Multi-stage	Receita em estágios: um estágio compila e outro só recebe o resultado. A imagem final fica menor e sem as ferramentas de build



Por que baixar o modelo no build

Se o modelo de embedding fosse baixado a cada início do serviço, cada reinício levaria mais tempo e dependeria do Hugging Face estar no ar. Baixado no build, ele vem dentro da imagem: o serviço sobe mais rápido e sem dependência externa.

10 Railway: o deploy de produção

No Railway, cada aplicação é um **serviço** dentro de um **projeto**. O nosso serviço nasce do repositório do GitHub e é montado pelo Dockerfile. O que não é segredo fica num arquivo do repositório, o `railway.json`. Isso é **infraestrutura como código**: a configuração é versionada, revisada e reproduzível como o resto do projeto.

```
{
  "build": { "builder": "DOCKERFILE", "dockerfilePath": "Dockerfile" },
  "deploy": { "healthcheckPath": "/api/saude",           // só troca a versão se responder
              "healthcheckTimeout": 300,
              "restartPolicyType": "ON_FAILURE" }      // reinicia sozinho se cair
}
```

Os recursos do Railway que usamos

Recurso	O que faz	Por que importa
Deploy from GitHub	Cria o serviço a partir do repositório e da branch	Cada commit vira uma versão
Variables	Variáveis de ambiente do serviço	Onde ficam as chaves (o segredo fica com quem executa)
Wait for CI	Só publica o commit depois que o GitHub Actions passa	O portão continua decidindo o que vai para o ar
Health check	Chama <code>/api/saude</code> antes de trocar a versão	Versão que não sobe não substitui a que está no ar
Logs e Metrics	Build, saída do servidor, CPU e memória	Diagnosticar erros e falta de memória
Rollback	Volta para um deploy anterior com um clique	Recuperação rápida

Por que ligar o Wait for CI

Sem ele, o Railway publica sozinho a cada commit e ignora o portão: um commit com a busca piorada iria para o ar. Ligado (Settings → Source), o Railway espera os jobs testar, avaliar e publicar terminarem verdes; se algum falhar, o deploy daquele commit é pulado e a versão anterior continua no ar.

Caso real: por que saímos do Render

O projeto de referência começou no Render. No plano de 512 MB, o build parou duas vezes com Out of memory (used over 512Mi), ainda no `pip install`. O Render limita o build à mesma memória do plano, e montar uma imagem com Node, Python e o modelo de embedding passa disso. Ajustar o Dockerfile (uma etapa de cada vez, modelo só baixado) não bastou. Havia três saídas: pagar o plano de 2 GB, montar a imagem no GitHub Actions ou trocar de plataforma. No Railway o build roda em máquinas separadas e passou em cerca de um minuto, com o mesmo Dockerfile.

A lição

Plataformas parecidas diferem nos limites que não aparecem na página inicial: memória do build, tempo de build, se o serviço dorme, como cobram. Leia os limites antes de escolher e mantenha o projeto portátil: com Dockerfile, trocar de plataforma custou poucos arquivos.

Limites e preços de planos mudam: confira no site de cada plataforma antes de escolher.

11 URL, HTTPS e domínio

Ao gerar um domínio (Settings → Networking → Generate Domain), o serviço ganha um endereço `<nome>.up.railway.app` com **HTTPS** automático: a conexão é cifrada por um certificado TLS que o Railway emite e renova sozinho. O nome pode ser editado no mesmo lugar.

Para quando quiser um domínio próprio

1. Compre um domínio (registro.br para .com.br) ou use um que você já tem.
2. No Railway: Settings → Networking → Custom Domain, por exemplo `chat.seudominio.com.br`.
3. No painel do seu domínio (DNS), crie os registros que o Railway mostrar: em geral um **CNAME** e, às vezes, um **TXT** de verificação.
4. O Railway verifica o DNS e emite o certificado HTTPS sozinho. Prefira um subdomínio (chat....): o domínio sem subdomínio exige um tipo de registro que nem todo DNS oferece.

12 PDF na base de conhecimento

Na Parte 2 a pasta documentos/ aceitava só Markdown, por dois motivos: o Hugging Face recusava binários e o PDF não guarda estrutura. Saindo do Hugging Face, o primeiro motivo some. O segundo continua, e agora quem resolve é o pipeline: o `indexar.py` converte cada PDF para Markdown antes de dividir em trechos.

Problema do PDF	O que a conversão faz
Títulos não marcados	<code>pymupdf4llm</code> identifica títulos pelo tamanho da fonte e gera <code>#</code> e <code>##</code>
Cabeçalho e rodapé em toda página	Linhas repetidas em mais da metade das páginas são removidas
Números de página soltos	Removidos
Tabelas	Convertidas em tabelas Markdown (as complexas podem sair imperfeitas)
PDF escaneado (só imagem)	Sem texto para extrair: a indexação para e avisa (OCR fica fora)

O portão protege a conversão

Conversão automática não garante qualidade. Por isso cada PDF novo deve ganhar perguntas em `perguntas_teste.yml`: se a extração vier ruim, a busca não acha o trecho, o hit rate cai e o deploy é barrado antes de chegar ao público.

Sobre licenças: o `pymupdf4llm` usa a licença AGPL. No projeto, ele roda só na indexação, no GitHub Actions, e não faz parte do servidor publicado. Uma alternativa com licença MIT é o `MarkItDown`, da Microsoft, que preserva menos a estrutura dos títulos.

13 Segurança e custo em produção

Risco	O que pode acontecer	Proteção no projeto
Chave no front-end	Qualquer visitante copia e usa a sua chave	Front-end só chama <code>/api</code> ; T17 procura chaves no build; nada de <code>VITE_</code> com segredo
Abuso da API	Alguém dispara milhares de perguntas e consome sua cota ou seu crédito	Limite de perguntas por minuto por visitante (429) e de tamanho (413)
Entrada maliciosa	Pedir para trocar o prompt ou o modelo	A API só aceita mensagem e histórico; prompt e modelo ficam no servidor
Injeção de prompt por documento	Um trecho com ordens para o modelo	Trechos delimitados e instrução para ignorar ordens dentro deles (Parte 2)
Vazamento em logs	Perguntas de usuários guardadas sem aviso	Logs só com tamanho, tempo e erro
Chave secreta do banco exposta	Alguém grava ou apaga a base	Servidor usa só a publicável; a secreta fica no GitHub

Por que o limite fica no servidor

Um limite feito só no front-end (desabilitar o botão, por exemplo) não protege nada: quem quiser abusar chama a API direto, sem passar pela interface. Toda regra de segurança precisa valer no servidor.

14 Observabilidade: saber o que está acontecendo

- **Health check** (`/api/saude`): diz se o servidor está pronto, qual versão está no ar (o commit, que o Railway informa em `RAILWAY_GIT_COMMIT_SHA`) e quantos trechos há na produção. O Railway só troca a versão quando esta rota responde 200.
- **Deploys**: cada commit aparece na aba Deployments com as etapas (build, deploy) e o status. Um commit barrado pelo portão aparece como pulado.
- **Logs**: build e servidor em tempo real. Mensagens claras (como "Base de conhecimento: modelo carregado; 178 trechos na produção") economizam horas.
- **Métricas**: uso de memória e CPU. Como o Railway cobra por uso, a métrica de memória também é a métrica de custo.

```
GET /api/saude
{"status": "ok", "versao": "60d70f4...", "trechos_na_producao": 178}

GET /api/saude (sem as variáveis do Supabase)
503 {"status": "indisponivel", ...} → o Railway não troca a versão
```

15 O projeto de referência por dentro

```
agente-teste/
├── frontend/
│   ├── src/components/
│   └── vite.config.js
├── servidor.py
├── chat.py
├── rag.py
├── indexar.py, avaliar.py
├── documentos/
├── testes.py, testes_api.py
├── Dockerfile
├── railway.json
└── .github/workflows/deploy.yml
```

React + Vite + Tailwind
 Cabecalho, Conversa, Mensagem, CartaoFonte, ...
 proxy de /api para a porta 8000 em desenvolvimento
 FastAPI: /api/saude, /api/config, /api/perguntar + front-end
 provedores de IA e geração da resposta (vem do app.py)
 agora lê .md e .pdf
 como na Parte 2
 .md e .pdf
 T1 a T17
 imagem em dois estágios
 o serviço do Railway como código
 testar → avaliar → publicar

Chave ou variável	Onde fica	Quem usa
SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY, chave do provedor	Variables do Railway	servidor.py
SUPABASE_URL, SUPABASE_SECRET_KEY	Secrets do GitHub	avaliar e publicar

O HF_TOKEN e o Space deixam de ser usados. O Space pode ser pausado ou apagado quando o Railway estiver no ar. Não há segredo novo no GitHub: quem liga o Railway ao portão é o Wait for CI.

16 Da ideia à spec da Parte 3

O processo é o mesmo das partes anteriores: o IDEIA-parte3.md em linguagem simples (Apêndice A), o prompt que está no fim dele para gerar a SPEC-parte3-frontend-producao.md, a revisão e as tarefas uma por vez. Na revisão, confira:

- Alguma chave aparece no front-end, no railway.json ou em variável VITE_?
- O Railway está com Wait for CI ligado, para só publicar o que passou no portão?
- Os limites de uso valem no servidor, não só na interface?
- O contrato da API lista cada rota, os dados e os erros?
- Os estados da interface (vazio, carregando, erro, não encontrei, limite) estão descritos?
- PDF escaneado tem comportamento definido?
- Os testes das Partes 1 e 2 continuam valendo, com os ajustes explícitos (T6 e T11)?

17 Passo a passo da aula

A. Preparar o Railway

1. Crie a conta em railway.com, entrando com o GitHub.
2. Autorize o Railway a ler o seu repositório.

B. Da ideia ao código

1. Crie o IDEIA-parte3.md a partir do Apêndice A e gere a spec com o prompt dele.
2. Revise pela lista da seção 16 e aprove.
3. Implemente uma tarefa por vez. Para o front-end, use o pedido de visual da seção 07 e itere até ficar com a sua cara.

C. Publicar

1. No Railway: New Project → Deploy from GitHub repo → o seu repositório, branch main.
2. Em Variables, cadastre OPENROUTER_API_KEY, SUPABASE_URL e SUPABASE_PUBLISHABLE_KEY.
3. Em Settings → Source, ligue Wait for CI. Em Settings → Networking, Generate Domain.
4. Faça um commit. Acompanhe os três jobs na aba Actions e, depois, o deploy no painel do Railway.

D. Testar

1. Abra o endereço no computador e no celular; alterne o tema.
2. Faça uma pergunta do material e abra os cartões de fonte.
3. Coloque um PDF com texto em documentos/, acrescente uma pergunta de teste sobre ele e faça commit.
4. Abra o código-fonte da página e procure qualquer coisa parecida com uma chave.

18 Erros comuns

Sintoma	Causa provável	Conserto
Build para com Out of memory (Render)	O build usa a memória do plano (512 MB)	Plano maior, build no GitHub Actions ou Railway
Railway usa "python" em vez do Dockerfile	Deploy de um commit sem Dockerfile/railway.json	Conferir a branch em Settings → Source
Railway publicou sem passar no portão	Wait for CI desligado	Ligar em Settings → Source
Deploy falha no health check	/api/saude responde 503	Ver os logs; conferir SUPABASE_URL e SUPABASE_PUBLISHABLE_KEY
Serviço no ar, mas sem endereço	Domínio não gerado (Unexposed service)	Settings → Networking → Generate Domain
Página em branco	dist ausente na imagem ou base errada no Vite	Conferir o estágio de build e base: "/"
Chat com erro de rede no computador	API local fora do ar ou proxy do Vite errado	Subir o uvicorn na porta 8000

Sintoma	Causa provável	Conserto
Chave no JavaScript publicado	Variável VITE_ com segredo	Remover, revogar a chave, manter só no servidor
PDF sem texto extraível	PDF escaneado	Usar um PDF com texto ou fazer OCR fora do projeto
Muitos 429 numa sala de aula	Todos atrás do mesmo IP da rede	Aumentar perguntas_por_minuto

19 Apêndice A: modelo de ideia da Parte 3

Resumo do IDEIA-parte3.md do projeto de referência. O arquivo completo, com o prompt para gerar a spec, está no repositório.

```
# Ideia do projeto · Parte 3: front-end próprio e deploy em produção

## O que eu quero
Dar ao assistente cara de produto: interface própria, bonita e moderna, num endereço próprio, com deploy mais robusto. E a base de conhecimento passa a aceitar PDF.

## Como eu quero que a interface seja
- Bonita e moderna, com as cores e a logo do config.yml; resposta aos poucos.
- Fontes como cartões clicáveis que mostram o trecho usado.
- Sugestões na tela inicial; celular; modo claro e escuro; tudo em português.

## Como eu quero que funcione por trás
- O navegador nunca recebe chave; um único endereço serve interface e API.
- Limites de tamanho e de perguntas por minuto, para ninguém gastar a minha chave.

## Documentos em PDF
- .pdf na pasta documentos/, convertido para Markdown na indexação.
- PDF escaneado: o portão avisa com mensagem clara.

## Onde eu quero publicar
- No Railway (professor-nta.up.railway.app), com HTTPS; configuração num arquivo do repositório.
- O GitHub Actions testa e avalia a busca; o Railway só publica se tudo passar.

## O que NÃO entra agora
- Login, conversas salvas, envio de documentos pela interface, domínio próprio, OCR.
```

20 Glossário

API: conjunto de rotas que um servidor responde, com um contrato de entrada e saída.

Back-end: a parte que roda no servidor: busca, chaves, chamada ao modelo.

Build: a etapa que transforma o código-fonte no que vai para produção (no Vite, a pasta dist).

CNAME: registro de DNS que aponta um nome para outro; usado no domínio próprio.

Componente: no React, uma função que devolve um pedaço da interface.

Contêiner: uma imagem Docker em execução.

CORS: regra do navegador para chamadas entre endereços diferentes; evitada com uma URL só.

Dockerfile: a receita de uma imagem Docker.

FastAPI: framework Python para APIs, com validação pelo Pydantic.

Front-end: a parte que roda no navegador: HTML, CSS e JavaScript.

Health check: rota que diz se o serviço está pronto; o Railway só troca a versão quando ela responde.

HTTPS / TLS: conexão cifrada por certificado; automática no Railway.

Imagem: pacote imutável com sistema, bibliotecas e código, gerado pelo Dockerfile.

Infraestrutura como código: configuração de servidores descrita em arquivos versionados.

Multi-stage: Dockerfile em estágios: um compila, outro só recebe o resultado.

PaaS: plataforma que executa sua aplicação cuidando da infraestrutura.

Pydantic: biblioteca que valida dados de entrada pelo tipo declarado.

Rate limit: limite de pedidos por tempo; protege custo e disponibilidade.

React: biblioteca JavaScript para interfaces em componentes.

Rollback: voltar para uma versão anterior do deploy.

Server-Sent Events: resposta HTTP aberta pela qual o servidor envia eventos ao navegador.

Status HTTP: código que resume o resultado de um pedido (200, 413, 422, 429, 503).

Tailwind CSS: biblioteca de classes utilitárias de estilo.

Vite: ferramenta de desenvolvimento e build de front-end.

Wait for CI: opção do Railway que só publica um commit depois que o GitHub Actions passa.